Region4Web: Rethinking Observation Space Granularity for Web Agents

Donguk Kwon
Yonsei University
donguk.kwon@yonsei.ac.kr

Dongha Lee *
Yonsei University
donalee@yonsei.ac.kr

Abstract

Web agents perceive web pages through an observation space, yet its granularity has remained an underexamined design choice. Existing work treats observation at the same element-level granularity as the action space, leaving the page's functional organization implicit and forcing the agent to infer it from element-level signals at every step. We argue observation should instead operate at the granularity of **functional regions**, parts of the page that each serve a distinct purpose. We propose **Region4Web**, a framework that reorganizes the AXTree into functional regions through hierarchical decomposition and semantic abstraction, exposing the page's functional organization as the basis for page state understanding. Moreover, we propose **PageDigest**, a web-specific inference pipeline that delivers this region-level observation to the actor agent as a compact per-page digest that persists across steps. On the WebArena benchmark, PageDigest substantially reduces observation length while improving overall task success rate across diverse backbone large language models (LLMs) and established agent methods, regardless of backbone capacity. These results show that operating at the granularity of functional regions delivers a more compact and informative basis for the actor agent than element-level processing alone. Code is available at `https://github.com/kwondu/region4web`.

1 Introduction

Large language models (LLMs) have enabled autonomous agents capable of handling diverse real-world tasks in web environments [13, 24, 41]. At each step, a web agent perceives the current page state through an observation space and selects an action from an action space. Prior work has concentrated on improving action selection, with task planning [12, 14, 32], element grounding [49], and model capability [30, 40] all directed toward this goal. Page state understanding, in contrast, has been addressed through filtering or truncating elements from the observation [15, 18, 47], which all operate at element-level granularity, leaving this design choice itself underexamined.

Existing work often represents the observation space at the same element-level granularity as the action space [31, 43], yet this granularity is not equally suited to both. Element-level granularity is natural for the action space, where each action targets a specific element with a designated operation. The observation space, however, serves a fundamentally different role of providing context for understanding the current page state, where context extends from individual elements to their relations. We capture these relations through **functional regions**, defined as groups of elements whose relations support a shared purpose, such as site traversal or result narrowing.

Decomposing pages into regions has been studied in human attention to spatially coherent areas [3] and recent GUI web agents that segment screenshots into region partitions [10, 33]. These approaches show that visual layout provides useful cues for grouping elements, often through spatial proximity such as bounding box overlap or layout adjacency. However, spatial proximity does not entail shared functional purpose. Such proximity cues may induce visual groupings, but do not specify

* Corresponding author

Preprint.

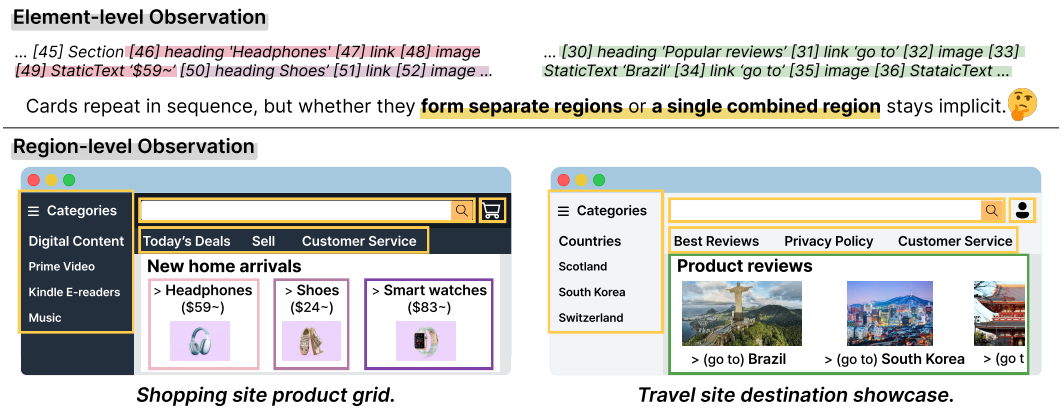


Figure 1: Element-level and region-level observation of structurally similar card grids. Region-level observation distinguishes a grid of product preview cards from a single destination showcase.

whether they constitute functional observation units or what purpose they serve in the page state. A similar implicitness appears in element-level observation [31, 43, 47], where regions and their purposes are present only implicitly through individual elements and must be inferred by the agent. Screenshot-based agents [13, 49] provide layout cues that may make functional organization visually inferable, but they still require the agent to infer whether visually suggested groupings correspond to functional regions and what purpose they serve. These limitations motivate region-level observation defined by shared functional purpose. By identifying functional regions and abstracting each by its purpose, Region4Web makes page organization explicit before action selection, as shown in Figure 1.

Constructing region-level observation is not straightforward. Boundaries and purposes of functional regions are implicit in tree representations such as AXTree, where the hierarchy reflects markup nesting rather than how elements are organized. Deriving them through rule-based decomposition is insufficient, as what each region is for varies with the page even for structurally repeated patterns. A grid of structurally repeated cards, for example, forms independent regions when the cards are separate product previews, yet a single region when they collectively form a review showcase, as Figure 1 demonstrates. Nor does existing research on web page structure resolve this, as web page segmentation [4, 11, 16] and content extraction [2, 22] methods target information retrieval or content analysis, not the functional organization that agent observation requires. Its construction therefore demands learning how web pages are functionally organized across diverse page layouts.

We address this challenge with **Region4Web**, a framework that constructs region-level observation from the AXTree through two stages. Hierarchical decomposition classifies each parent-child edge as merge or cut in a single bottom-up traversal, and the subtrees formed by merged edges constitute the functional regions of the page. Semantic abstraction then interprets each region along two orthogonal dimensions, a purpose that identifies what the region is for and a state summary that captures its current actionable context. Since both stages run at every page during agent execution, they are realized as small dedicated models. The knowledge of how pages are functionally organized is implicit in the AXTree and cannot be derived by rule, so these models are trained on annotations from a proprietary LLM covering diverse real-world websites.

Moreover, deploying Region4Web in web environments requires keeping its region-level observation compact while preserving the page state understanding it supports, which motivates **PageDigest**, a web-specific inference pipeline that maintains a compact digest of the agent's observation across steps within each page. Upon entering a new page, PageDigest selects task-relevant regions and exposes them as AXTree subtrees alongside the non-selected regions' abstractions, preserving element-level granularity for the action space within the page's structural information. Within the same page, PageDigest tracks observation transitions across steps, rather than reconstructing the full observation at every step. PageDigest shares the actor agent's backbone LLM and operates solely on the observation space, making it directly applicable to diverse web agents.

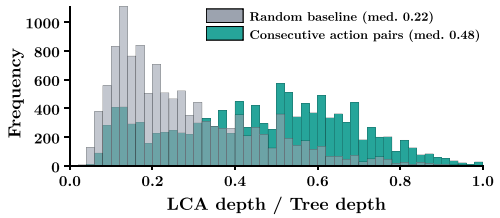
On the WebArena [50] benchmark, PageDigest substantially reduces observation length across four backbone LLMs and two established agent methods, with the reduction holding consistently regardless of backbone capacity. PageDigest improves overall task success rate across backbones, demonstrating that region-level observation strengthens page state understanding regardless of backbone capacity.

Ablations confirm that Region4Web and PageDigest make distinct contributions, with Region4Web alone supporting page state understanding while PageDigest delivers it compactly across steps.

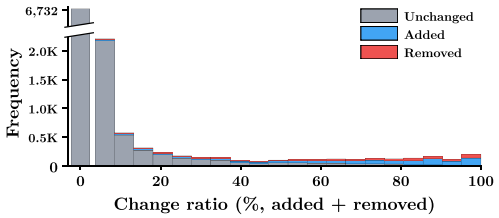
Our contributions are summarized as follows.

- We propose Region4Web, a framework that reorganizes the AXTree into functional regions through hierarchical decomposition and semantic abstraction, exposing the page's functional organization as the basis for web agents' page state understanding.
- We propose PageDigest, a web-specific inference pipeline that delivers each page's region-level observation to the actor agent as a compact digest that persists across steps, reducing observation length while preserving task success.
- We evaluate Region4Web and PageDigest on the WebArena benchmark, where PageDigest substantially reduces observation length while improving overall task success rate, regardless of backbone capacity.

2 Preliminary Analysis



(a) Distribution of LCA depth ratio for consecutive action pairs against the random baseline.



(b) Distribution of DOM change ratio across within-page steps, where 52.9% exhibit zero change.

We analyze action traces and observation transitions to inform two design questions about observation in web environments. Section 2.1 examines whether the agent's actions are localized within the page structure during a task, motivating the unit at which observation should be constructed within a single step. Section 2.2 examines how much the observation changes as the agent acts within a page, motivating the question of whether observation should be reconstructed at every step.

To answer these questions, we use the Mind2Web dataset [8], which provides 2,350 tasks with per-action ground-truth annotations across 137 real-world websites, with dataset selection criteria detailed in Appendix C. Each page is represented as a DOM tree with an average of 2,473 nodes. The dataset contains 15,394 consecutive action pairs, of which 12,009 (78.0%) occur within the same page and the remaining 22.0% involve page navigation that entirely replaces the observation. Our analysis focuses on same-page pairs, where observation construction and update are at issue.

2.1 Consecutive Actions Are Localized within Page Structure

Only a negligible fraction of elements on a page are targeted during a task. While each page contains thousands of DOM nodes, the number of actions performed on it during a task has a median of 6 and a 90th percentile of 13. Since each action targets exactly one element, the elements ever acted upon constitute a negligible fraction of the page. The full page is thus dominated by elements irrelevant to the task, motivating selection of task-relevant content.

Consecutive actions are structurally co-located within the page. We measure the lowest common ancestor (LCA) depth ratio for consecutive action pairs, computed as the depth of the LCA of the two target elements divided by the maximum depth of the DOM tree. A higher value indicates that the two elements are situated within a tighter subtree. As Figure 2a shows, consecutive action pairs yield a median LCA depth ratio of 0.48, with 81.7% exceeding the random baseline median of 0.22. Consecutive actions thus concentrate within localized subtrees rather than spanning the page, indicating that the region serves as a natural unit for observation construction.

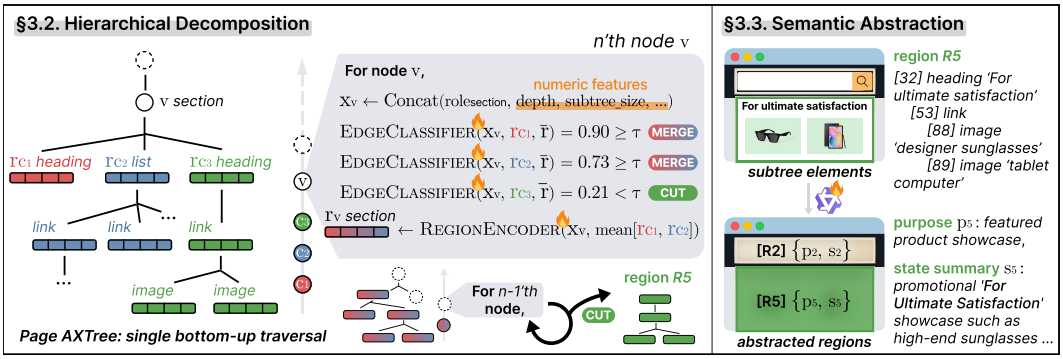


Figure 3: Overview of Region4Web inference process.

2.2 Within-Page Observation Undergoes Marginal Change across Consecutive Steps

For each step within a page, we measure the change ratio, the proportion of DOM elements added or removed by the action. As Figure 2b shows, 52.9% of steps exhibit zero change, and 74.4% remain below 5%. Where changes occur, they reflect minor DOM modifications such as dropdown expansion or tooltip appearance. Steps exceeding 90% change account for only 2.5%, attributed to client-side routing within single-page applications. Reconstructing the full observation at every step is therefore unnecessary, and tracking only the incremental changes within each page can avoid this redundancy.

3 Region4Web

Section 2.1 shows that regions are natural units for observation. We propose Region4Web, a two-stage framework for constructing region-level observation from the AXTree of a web page.

3.1 Problem Formulation

At each step, a web agent perceives the current page state through an observation space and selects an action from an action space. The observation can be represented as a tree $\mathcal{T} = (V, E)$, where each node $v \in V$ corresponds to an element on the page with attributes such as role, name, and value. In the prevailing element-level approach, the agent operates over V directly, leaving the page's functional organization implicit in $\mathcal{T}$. Region-level observation makes this organization explicit through a partition $\mathcal{R} = \{R_1, \ldots, R_m\}$ of V into functional regions, where each R_i forms a subtree of $\mathcal{T}$. Each region is associated with a purpose p_i that identifies what the region is for and a state summary s_i that captures its current actionable context. Region4Web learns to produce both $\mathcal{R}$ and the associated $\{(p_i, s_i)\}$ from $\mathcal{T}$.

3.2 Hierarchical Decomposition

To construct region-level observation, $\mathcal{T}$ must be decomposed into a region partition $\mathcal{R}$. We instantiate $\mathcal{T}$ as the page's AXTree, a browser-generated representation that encodes each element's accessibility semantics in a hierarchical structure. Since each $R_i \in \mathcal{R}$ forms a subtree of $\mathcal{T}$, the partition is fully determined by classifying each edge in E as merge or cut. Removing the cut edges from $\mathcal{T}$ splits the tree into subtrees, each of which constitutes a region in $\mathcal{R}$. Since the root has no parent edge to classify, its subtree constitutes the final region in $\mathcal{R}$ after the bottom-up traversal completes.

Decomposition determines region boundaries from structural cues alone, whereas semantic abstraction interprets each region's purpose and actionable state. Each node v is represented by a feature vector $\mathbf{x}_v$ that combines a learned role embedding with numeric features encoding the node's structural information in $\mathcal{T}$. At each internal node v with children $c_1, \ldots, c_k$ and their respective representations $\mathbf{r}_{c_1}, \ldots, \mathbf{r}_{c_k}$, an EdgeClassifier determines whether each child should be separated, using the sibling mean $\bar{\mathbf{r}} = \frac{1}{k}\sum_j \mathbf{r}_{c_j}$ as context,

$$\hat{y}_{v,c_i} = \textsc{EdgeClassifier}(\mathbf{x}_v,\ \mathbf{r}_{c_i},\ \bar{\mathbf{r}}). \tag{1}$$

Edges with $\hat{y}_{v,c_i} \geq \tau$ are cut, while the remaining children $\mathcal{M}_v$ are merged into the parent's region. RegionEncoder then computes the parent's representation from $\mathbf{x}_v$ and the merged children $\mathcal{M}_v$,

$$\mathbf{r}_v = \textsc{RegionEncoder}\left(\mathbf{x}_v,\ \frac{1}{|\mathcal{M}_v|}\sum_{c_j \in \mathcal{M}_v} \mathbf{r}_{c_j}\right), \tag{2}$$

ensuring that the parent's representation reflects only the children that belong to its region. For leaf nodes, since no children exist, $\mathcal{M}_v$ is empty and the aggregation term reduces to $\mathbf{0}$.

The entire procedure is carried out in a single bottom-up traversal, where each node's representation is computed only after all its children's boundary decisions are resolved, so that boundary decisions propagate upward through the hierarchy without requiring an additional pass. The full procedure is detailed in Algorithm 1.

3.3 Semantic Abstraction

The region partition $\mathcal{R}$ determines which elements belong together, but the semantic meaning of each region remains implicit in its subtree. A fine-tuned language model receives the preprocessed AXTree subtree of each region and produces a purpose p_i and a state summary s_i, which address two orthogonal dimensions. Purpose captures what the region is for, serving as the basis for identifying each region. State summary interprets the region's current actionable context, conveying what information and actionable elements are available within it.

3.4 Training

Since both stages run on every page during agent execution, they are realized as small dedicated models, a decomposition model for structural boundary decisions and a small language model for per-region abstraction. The knowledge of how web pages are functionally organized is implicit in $\mathcal{T}$ and cannot be derived by rule, so these models are trained on annotations from a proprietary LLM. We employ `gpt-5-mini-2025-08-27` [28] as the annotator to construct the training dataset. The raw AXTree is preprocessed into a textual form that retains the elements an agent can perceive and act on along with the structural grouping among them. Since Region4Web operates sequentially, with decomposition producing $\mathcal{R}$ that abstraction then interprets, the training dataset should be constructed to follow this same dependency.

Dataset Construction. Source pages are collected from 500 real-world websites sampled from the Tranco top-1M ranking list, [2] a research-oriented ranking of most popular websites. These websites span 10 domain categories (e.g., Technology & Computing, Shopping) derived from the IAB Content Taxonomy, [3] a standard classification of web content, for their relevance to web agent tasks. For each website, up to 100 page URLs are sampled using a score computed from sitemap metadata, yielding 21,974 pages from 253 websites whose AXTrees are successfully extracted. The annotator then processes each page in three steps. It first decomposes the AXTree into a region partition, then verifies the partition to identify incorrectly formed regions, and finally produces a purpose and a state summary for each verified region. Only pages whose partitions contain no invalid region are retained, yielding 2,052 pages and 45,147 regions. Pages excluded by this filter are dominated by real-world website noise that prevents coherent region organization rather than by annotator capacity, so the retained pages carry reliable annotations.

Decomposition training. The verified region partitions are converted into binary edge labels over E, where each edge is labeled as cut if its parent and child belong to different regions and as merge otherwise. The model is trained with teacher forcing, where ground-truth labels determine the cut and merge decisions during the bottom-up traversal so that each node's representation is computed from correctly partitioned children. Since merge edges vastly outnumber cut edges, focal loss with $\alpha = 0.75$ and $\gamma = 2.0$ is applied to address the class imbalance [20, 25].

Abstraction training. Qwen3-0.6B [36] is fine-tuned on the 45,147 region annotations from the verified pages, with each example pairing a region's preprocessed subtree as input with the corresponding purpose and state summary as output. A small model is chosen so that abstraction can be invoked once per region without dominating inference latency.

[2] Tranco top-1M ranking list snapshot from April 1, 2026. `https://tranco-list.eu/list/QWQ94/1000000`

[3] IAB Content Taxonomy 3.1. `https://iabtechlab.com/standards/content-taxonomy`

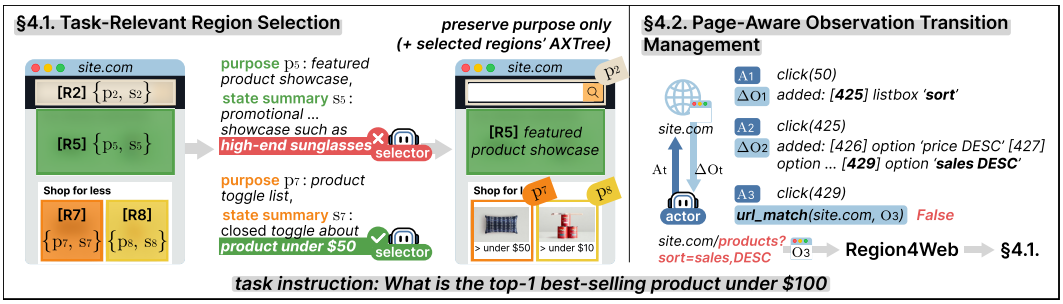


Figure 4: Overview of PageDigest.

Further details on AXTree preprocessing, dataset construction, and Region4Web implementation are provided in Appendices D, E, and F, respectively.

4 PageDigest

Region4Web produces region-level observation for a given page, but deploying it in web environments requires focusing the observation on what is task-relevant and tracking how pages change as the agent acts. We propose PageDigest, a web-specific inference pipeline that constructs a page digest upon entering a new page through region selection, retains it within the page, and updates it through observation transition tracking across steps.

4.1 Task-Relevant Region Selection

Upon entering a new page, Region4Web produces the region partition $\mathcal{R}$ and the associated $\{(p_i, s_i)\}$ for the page. The actor agent's backbone LLM takes the abstractions $\{(p_i, s_i)\}$ together with the task instruction and the action history taken so far, and selects the task-relevant regions. The abstractions specify each region individually and collectively convey the page's overall functional structure and current state, from which the model infers where the task currently stands and what is required next. Selected regions are exposed to the actor agent as their AXTree subtrees with their purposes, preserving element-level granularity for the action space, while non-selected regions are represented by their purposes alone, retaining the page's overall structural information.

4.2 Page-Aware Observation Transition Management

Section 2.2 shows that within-page observation changes only marginally between consecutive steps. PageDigest therefore tracks observation transitions across steps, rather than reinvoking Region4Web at each step. During transition management, only the region purposes are referenced, since each purpose describes what its region is for and remains stable across steps within the page. State summaries, in contrast, describe each region's current actionable context, making them useful for region selection at page entry but less suitable for page-aware observation transition management.

At each step, observation transitions are identified by comparing the current AXTree against its state at page entry, yielding added, removed, and modified nodes. Removed and modified nodes update the AXTree constructed upon entering the page by deleting nodes or changing their values under the existing region purposes. Added nodes, in contrast, are listed as a separate group that retains the structural grouping among them, since merging them into existing regions could shift those regions' purposes. The actor agent thus receives the current observation across steps within the page, preserving continuity of the page state. When the agent navigates to a new page, signaled by a URL change, Region4Web is invoked to produce the new page's $\mathcal{R}$ and $\{(p_i, s_i)\}$, and region selection proceeds.

PageDigest shares the actor agent's backbone LLM and operates solely on the observation space, requiring no additional model and leaving the actor agent's policy unmodified, making it directly applicable to diverse web agents. Moreover, since region selection is performed by the actor agent's backbone and depends on its capability, the actor agent is given additional `view_all` action that reveals all regions in their full AXTree subtree form for the remainder of the page, providing a fallback when the selected regions are insufficient.

Table 1: WebArena success rate (%) across domains, with the average observation token length per step reported in the Obs. length column. Each actor agent is reported with and without PageDigest.

Actor agent	Shopping	CMS	Reddit	GitLab	Map	Overall	Obs. length (Δ)
Proprietary LLMs							
GPT-5.1	39.0	**57.1**	**65.1**	46.1	24.8	45.2	6,116
+ PageDigest	**41.1**	54.5	60.5	**50.5**	**28.4**	**47.5**	**4,302 (-30%)**
Gemini 3.1 Flash-Lite	33.3	41.2	54.0	**44.7**	22.9	39.4	6,705
+ PageDigest	**34.9**	**44.5**	**59.3**	42.6	**28.4**	**41.6**	**3,207 (-52%)**
Open-source LLMs							
DeepSeek-V3.2	30.7	51.1	58.5	40.4	**21.1**	40.4	7,158
+ PageDigest	**33.3**	**53.7**	**61.1**	**46.5**	**21.1**	**43.4**	**4,521 (-37%)**
Qwen3.5-27B	22.9	**53.8**	58.8	**35.3**	**22.9**	38.2	5,767
+ PageDigest	**38.6**	46.2	**60.4**	35.2	18.9	**39.9**	**2,654 (-54%)**
Web agent methods (backbone: GPT-4o)							
SteP [34]	32.6	**45.7**	**71.4**	46.9	12.8	**39.5**	7,136
+ PageDigest	**35.8**	37.1	63.2	**50.0**	**15.4**	38.7	**3,693 (-50%)**
AgentOccam [43]	26.7	36.3	**73.7**	**66.7**	11.5	40.6	4,025
+ PageDigest	**35.6**	**40.0**	68.4	50.0	**23.1**	**41.3**	**3,365 (-16%)**

5 Experiments

5.1 Experimental Setup

Evaluation benchmark. We evaluate on WebArena [50], a comprehensive web agent benchmark that spans five distinct domains, namely e-commerce, social forum, collaborative development, content management, and map services. Its 812 long-horizon tasks, each allowing up to 30 steps, cover diverse interaction patterns. Since the original evaluator relies on `gpt-4-1106-preview` for fuzzy answer matching, which has since been deprecated, we replace it with GPT-4o.

Actor agents. We evaluate across diverse backbone LLMs and actor agent methods to verify that PageDigest consistently reduces observation length regardless of the model's capability or the agent's design while preserving the performance. The backbone LLMs span two proprietary and two open-source LLMs, namely GPT-5.1 [28], Gemini 3.1 Flash-Lite [1], Deepseek-V3.2 [7], and Qwen3.5-27B [36]. Each backbone selects the next action given the interaction history and current observation at each step [44]. We further evaluate on two established agent methods widely adopted in WebArena evaluation. SteP [34] dynamically composes human-designed LLM policies tailored to WebArena tasks through a stack-based Markov decision process. AgentOccam [43] refines the observation and action spaces to align them with the underlying LLM's pretrained capabilities. Since AgentOccam runs with its own space alignment, in the PageDigest configuration, we replace its alignment with PageDigest while retaining the action space alignment, isolating the effect of region-level observation. We evaluate SteP and AgentOccam with GPT-4o as the backbone, matching the GPT-4 family under which both methods were originally developed.

Implementation details. All experiments are conducted in the BrowserGym environment, with Map domain tasks routed to the live OpenStreetMap service [4] following [5, 48]. For reproducibility, open-source models are run at temperature 0 with thinking mode disabled where applicable, while proprietary models retain their default configuration. We define observation length as the token count of the observation provided at the agent at each step. All token counts reported in the experiments are measured under the OpenAI `o200k_base` tokenizer. All prompts are provided in Appendix H.

5.2 Main Results

PageDigest improves overall task success rate while reducing observation length, regardless of backbone capacity. Across the four backbones in Table 1, PageDigest reduces observation length by 43% on average, from 6,437 to 3,671 tokens, and improves task success rate by 2.3%p on

[4] OpenStreetMap service. `https://www.openstreetmap.org`

average. The improvement holds across backbones of varying capacity, suggesting that region-level observation provides a complementary signal for page state understanding that benefits backbones independent of their strength.

PageDigest extends to established agent methods through the observation space. Applying PageDigest to SteP and AgentOccam reduces observation length by 50% and 16% with comparable task success rate. For AgentOccam, replacing its observation space alignment with PageDigest yields comparable performance, showing region-level observation can replace element-level alignment for action selection. Since PageDigest operates solely on the observation space, sharing the actor's backbone, it applies to diverse web agents, with task success scaling with backbone capacity.

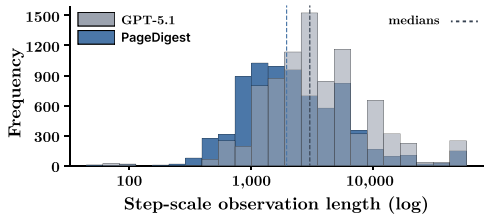
5.3 Further Analysis

We further analyze the contributions of Region4Web and PageDigest using GPT-5.1, with case studies of Region4Web's decomposition and abstraction in Appendix G.

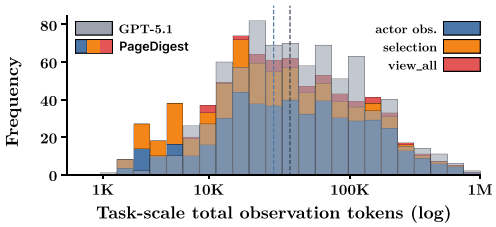
Region4Web improves page state understanding, while PageDigest keeps it compact across steps. For the ablation study, we use WebArena-Lite [18, 23], a 165-task subset of WebArena. Table 2 compares the backbone, Region4Web alone, an element-level variant of PageDigest that omits Region4Web and replaces the region selection stage with self-contextualization as in LCoW [18], and PageDigest. Region4Web alone improves task success rate from 48.5% to 50.3% with comparable observation length, while the element-level variant lowers it to 46.1%, showing that region-level observation supports the actor agent where element-level processing instead hinders it. PageDigest reduces observation length by 30%, comparable to the element-level variant's 26% reduction, while still achieving the highest task success rate among the configurations, improving over the backbone by 5.4%p. Together, these results show that page state understanding and compact persistence are complementary, with Region4Web preserving functional regions for page state understanding and PageDigest keeping them compact across steps.

Table 2: Ablation on WebArena-Lite. The Obs. length column matches Table 1.

Configuration	SR (%)	Obs. length
GPT-5.1	48.5	5,410
+ Region4Web	<u>50.3</u>	5,922
+ Self-ctx [18] + § 4.2	46.1	<u>4,013</u>
+ PageDigest	**53.9**	**3,814**



(a) Distribution of step-scale observation length.



(b) Distribution of task-scale total observation tokens.

PageDigest preserves its step-scale reduction at the task-scale despite the auxiliary inference it adds. As Figure 5a shows, PageDigest reduces the median observation length by 33%, from 3,077 to 2,066 tokens, and as Figure 5b shows, the median cumulative observation across a task drops by 25%, from 26,707 to 19,944 tokens. The task total comprises more than the actor observations alone, since region selection inputs are added at each entry into a new page, and `view_all` expansions are added whenever the fallback is triggered. Decomposing the task total, the actor observation accounts for 73.9%, region selection for 19.5%, and `view_all` for 6.6%. Region selection stage stays cheap since it operates over the region-level abstractions $\{(p_i, s_i)\}$ rather than the element-level AXTree, even when invoked 4.8 times on average across a task, and `view_all` is invoked sparingly in only 38.1% of tasks, with an average of 0.64 calls across a task. The auxiliary overhead therefore stays bounded by page entries, and the step-scale compactness carries through to the task-scale.

PageDigest's failures lie largely outside its own design. We randomly sample 50 failed task trajectories under PageDigest on WebArena, 10 from each domain, and trace the failure to its

triggering step and label every PageDigest stage, as shown in Figure 6. Decomposition and abstraction errors (each 2.0%) together account for only a small fraction, indicating that Region4Web reliably decomposes and abstracts regions on most pages. Selection errors (10.0%) reflect the backbone LLM missing task-relevant regions despite Region4Web's informative abstractions. Transition management introduces no errors, as it deterministically compares the current AXTree against its state at page entry. Actor-side failures in the backbone's action selection account for 90.0%, with environment errors outside the pipeline adding 16.0%. PageDigest thus operates as designed, with 82.0% backbone capacity dominating 8.0% PageDigest regression under multi-cause attribution.

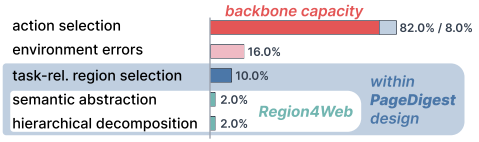


Figure 6: Failure mode distribution under PageDigest on WebArena.

6 Related Work

Web Page Structure Understanding has been studied for information retrieval and content analysis, treating web pages as content to be processed rather than as observation for agents. Web page segmentation partitions pages into visually or structurally coherent blocks, exemplified by VIPS [4]. Subsequent work has focused on evaluation methodology [16, 17] and macro-structural labels such as header, main content, and footer [11]. Content extraction separates main content from surrounding noise through rule-based heuristics [2] or language models [6, 22, 37]. In contrast, our Region4Web constructs region-level observation for web agents by decomposing the page into functional regions and making each region's purpose explicit for action selection.

Observation Processing in Web Agents has explored strategies to reduce observation length while preserving task-relevant information. A dominant line focuses on element selection [26], where Prune4Web [47] filters elements via LLM-generated keyword matching programs, and LCoW [18] trains a contextualization module that extracts task-relevant elements and annotates them contextually. Orthogonal to selection, Beyond Pixels [31] downsamples the DOM tree while preserving its hierarchical structure. AgentOccam [43] reformulates elements into markdown and identifies pivotal nodes to retain across steps. Multimodal web agents use screenshots as additional input [12, 13, 49], while recent GUI agents introduce visually decomposed region structures [10, 33]. These approaches provide visual or layout cues for understanding the page state, but they do not define observation units by shared functional purpose, leaving which elements form functional regions and what purposes those regions serve implicit. Our work treats observation granularity as a design choice, shifting from element-level to region-level observation and deploying it through a web-specific inference pipeline.

Tree-Structured Representation Learning has been studied across domains, from syntactic parse trees in natural language processing [35], to abstract syntax trees in source code analysis [27, 39, 46], to DOM trees in web page understanding [38, 45]. These methods typically compute representations over a fixed tree structure and use the resulting node or tree representations for downstream prediction. In this design, the tree structure is given in advance, and representation learning does not change which children belong to each parent. Region partitioning breaks this independence, as boundary decisions directly alter the set of children a parent must represent. This boundary-representation dependency motivates the joint computation in a single bottom-up traversal that Region4Web adopts.

7 Conclusion

We presented Region4Web and PageDigest, addressing observation granularity as an underexamined design choice for web agents. Region4Web reorganizes the AXTree into functional regions to support the actor agent's page state understanding, and PageDigest delivers this region-level observation as a compact digest that persists across steps. On the WebArena benchmark, PageDigest substantially reduces observation length while improving overall task success rate across diverse backbone LLMs and established agent methods, demonstrating that region-level observation can provide a more compact and informative basis for web agent decision making than element-level processing. These results show that observation granularity directly affects web agent efficiency. By separating observation design from model capability and action policy, our work opens a path toward more efficient web agents by rethinking the granularity at which pages are observed.

References

[1] Google AI. Gemini 3.1 flash-lite: Built for intelligence at scale, 2026.

[2] Adrien Barbaresi. Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2021.

[3] Georg Buscher, Edward Cutrell, and Meredith Ringel Morris. What do you see when you're surfing?: using eye tracking to predict salient regions of web pages. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2009.

[4] Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. Vips: a vision-based page segmentation algorithm, 2003.

[5] Hyungjoo Chae, Namyoung Kim, Kai Tzu iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, 2025.

[6] Yihan Chen, Benfeng Xu, Xiaorui Wang, and Zhendong Mao. An index-based approach for efficient and effective web content extraction, 2025.

[7] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025.

[8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems*, 2023.

[9] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? In *The Forty-First International Conference on Machine Learning*, 2024.

[10] Yue Fan, Lei Ding, Ching-Chen Kuo, Shan Jiang, Yang Zhao, Xinze Guan, Jie Yang, Yi Zhang, and Xin Eric Wang. Read anywhere pointed: Layout-aware gui screen reading with tree-of-lens grounding. In *The 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.

[11] Jonathan Gerber, Jasmin Saxer, Kimia Rabishokr, Bruno Kreiner, and Andreas Weiler. Webclasseg-25: A dual-classified webpage segmentation dataset. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025.

[12] Yuhan Guo, Cong Guo, Aiwen Sun, Hongliang He, Xinyu Yang, Yue Lu, Yingji Zhang, Xuntao Guo, Dong Zhang, Jianzhuang Liu, Jiang Duan, Yijia Xiao, Liangjian Wen, Hai-Ming Xu, and Yong Dai. Web-cogreasoner: Towards knowledge-induced cognitive reasoning for web agents. In *International Conference on Learning Representations*, 2026.

[13] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[14] Peng Huang, Xin Zheng, Jiayi Lin, Yuxiang Zhang, Jingkai Zhou, Zhicheng Yang, Ruibin Yuan, Zhenghao Liu, Yukun Yan, Ge Zhang, and Wenhao Huang. R2d2: Remembering, reflecting and dynamic decision making for web agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[15] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. Acon: Optimizing context compression for long-horizon llm agents, 2025.

[16] Johannes Kiesel, Lars Meyer, Florian Kneist, Benno Stein, and Martin Potthast. Web page segmentation revisited: Evaluation framework and dataset. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, 2020.

[17] Johannes Kiesel, Lars Meyer, Florian Kneist, Benno Stein, and Martin Potthast. An empirical comparison of web page segmentation algorithms. In *Proceedings of the 43rd European Conference on IR Research*, 2021.

[18] Dongjun Lee, Juyong Lee, Kyuyoung Kim, Jihoon Tack, Jinwoo Shin, Yee Whye Teh, and Kimin Lee. Learning to contextualize web pages for enhanced decision making by llm agents. In *International Conference on Learning Representations*, 2025.

[19] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. Pytorch distributed: experiences on accelerating data parallel training. *Proc. VLDB Endow.*, 2020.

[20] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, 2017.

[21] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *International Conference on Learning Representations*, 2018.

[22] Mengjie Liu, Jiahui Peng, Wenchang Ning, Pei Chu, Jiantao Qiu, Ren Ma, He Zhu, Rui Min, Lindong Lu, Linfeng Hou, Kaiwen Liu, Yuan Qu, Zhenxiang Li, Chao Xu, Zhongying Tu, Wentao Zhang, and Conghui He. Dripper: Token-efficient main html extraction with a lightweight lm, 2025.

[23] Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Yifan Xu, Xixuan Song, Shudan Zhang, Hanyu Lai, Xinyi Liu, Hanlin Zhao, Jiadai Sun, Xinyue Yang, Yu Yang, Zehan Qi, Shuntian Yao, Xueqiao Sun, Siyi Cheng, Qinkai Zheng, Hao Yu, Hanchen Zhang, Wenyi Hong, Ming Ding, Lihang Pan, Xiaotao Gu, Aohan Zeng, Zhengxiao Du, Chan Hee Song, Yu Su, Yuxiao Dong, and Jie Tang. Visualagentbench: Towards large multimodal models as visual foundation agents. In *International Conference on Learning Representations*, 2025.

[24] Lajanugen Logeswaran, Jaekyeom Kim, Sungryull Sohn, Creighton Glasscock, and Honglak Lee. Scaling web agent training through automatic data generation and fine-grained evaluation. In *Second Conference on Language Modeling*, 2025.

[25] Yihong Ma, Yijun Tian, Nuno Moniz, and Nitesh V. Chawla. Class-imbalanced learning on graphs: A survey. *ACM Computing Survey*, 2025.

[26] Anita Moskaleva, Mohamed Abdelhady, Angelos Katharopoulos, Daniel Toyama, and Simon Schug. Focusagent: Simple yet effective ways of trimming the large context of web agents, 2025.

[27] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. Convolutional neural networks over tree structures for programming language processing. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016.

[28] OpenAI. Openai gpt-5 system card, 2025.

[29] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. Webcanvas: Benchmarking web agents in online environments, 2024.

[30] Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, 2025.

[31] Thassilo M. Schiepanski and Nicholas Piël. Beyond pixels: Exploring dom downsampling for llm-based web agents, 2025.

[32] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[33] Kunal Singh, Shreyas Singh, and Mukund Khanna. Trishul: Towards region identification and screen hierarchy understanding for large vlm based gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2025.

[34] Paloma Sodhi, S. R. K. Branavan, Yoav Artzi, and Ryan McDonald. Step: Stacked llm policies for web actions. In *First Conference on Language Modeling*, 2024.

[35] Kai Sheng Tai, Richard Socher, and Christopher D. Manning. Improved semantic representations from tree-structured long short-term memory networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing*, 2015.

[36] Qwen Team. Qwen3 technical report, 2025.

[37] Feng Wang, Zesheng Shi, Bo Wang, Nan Wang, and Han Xiao. Readerlm-v2: Small language model for html to markdown and json, 2025.

[38] Qifan Wang, Yi Fang, Anirudh Ravula, Fuli Feng, Xiaojun Quan, and Dongfang Liu. Webformer: The web-page transformer for structure information extraction. In *Proceedings of the ACM Web Conference 2022*, 2022.

[39] Wenhan Wang, Ge Li, Sijie Shen, Xin Xia, and Zhi Jin. Modular tree network for source code representation learning. *ACM Transactions on Software Engineering and Methodology*, 2021.

[40] Zhepei Wei, Wenlin Yao, Yao Liu, Weizhi Zhang, Qin Lu, Liang Qiu, Changlong Yu, Puyang Xu, Chao Zhang, Bing Yin, Hyokun Yun, and Lihong Li. Webagent-r1: Training web agents via end-to-end multi-turn reinforcement learning. In *The 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

[41] Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[42] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents. In *Second Conference on Language Modeling*, 2025.

[43] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. Agentoccam: A simple yet strong baseline for llm-based web agents. In *International Conference on Learning Representations*, 2025.

[44] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[45] Benedict Yeoh and Huijuan Wang. Grown+up: A graph representation of a webpage network utilizing pre-training. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022.

[46] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. A novel neural source code representation based on abstract syntax tree. In *Proceedings of the 41st International Conference on Software Engineering*, 2019.

[47] Jiayuan Zhang, Kaiquan Chen, Zhihao Lu, Enshen Zhou, Qian Yu, and Jing Zhang. Prune4web: Dom tree pruning programming for web agent. In *Proceedings of the 40th AAAI Conference on Artificial Intelligence*, 2026.

[48] Weiming Zhang, Jihong Wang, Jiamu Zhou, Qingyao Li, Xinbei Ma, Congmin Zheng, Xingyu Lou, Weiwen Liu, Zhuosheng Zhang, Jun Wang, Yong Yu, and Weinan Zhang. Plan-mcts: Plan exploration for action exploitation in web navigation, 2026.

[49] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded. In *The Forty-First International Conference on Machine Learning*, 2024.

[50] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.

A Limitations and Future Work

Region4Web operates over the AXTree, and its decomposition and abstraction quality therefore depend on how completely each page exposes its accessibility semantics, with pages that render through canvas or rely on non-semantic markup providing weaker structural cues for boundary classification. Our evaluation focuses on WebArena, whose consistent AXTree fidelity supports the controlled comparisons our experiments require. Broader validation on real-world web environments, where AXTree fidelity and page complexity vary across sites, complements these results. Future work includes broadening evaluation to live web environments such as Online-Mind2Web [42], and applying region-level granularity to screenshot-based agents, since organizing observation by shared functional purpose generalizes across modalities.

B Broader Impacts

Region4Web and PageDigest reduce the observation length required for web agent operation, which lowers inference cost and broadens access to web agent technology in resource-constrained settings. The same efficiency gains can also lower the barrier to misuse such as large-scale scraping or automated abuse of online services, where mitigation lies at the deployment level through controls such as rate limiting and access policies. Our training data is constructed from publicly accessible pages on Tranco-listed websites and contains no personal or sensitive information, limiting privacy concerns from the released artifacts.

C Dataset Selection for Preliminary Analysis

Our preliminary analysis requires a web agent benchmark that provides per-action ground-truth annotations across diverse real-world web pages, so that consecutive action targets can be identified and localized within the page structure. Mind2Web [8] is well suited for this purpose. It provides 2,350 tasks across 137 websites spanning 31 domains, where each action step is grounded in the DOM snapshot of the page at that step. Since our analysis targets structural properties within individual snapshots, the static nature of these representations does not affect the validity of the measurements.

Other web agent benchmarks do not meet these requirements. MiniWoB++ [21] consists of atomic-level tasks in synthetic web environments that do not reflect the structural complexity of real-world pages. Mind2Web-Live [29] provides tasks on live websites, but its annotations adopt a key-node evaluation scheme that assesses task completion at designated milestones rather than providing per-action ground-truth annotations with element-level targets. Although the raw data provides per-action ground-truth annotations, [5] page sources are identified by URL without stored snapshots, and the referenced pages have since undergone content updates and layout modifications, making the original page structures unrecoverable.

D AXTree Preprocessing

Our AXTree preprocessing follows BrowserGym [9], which extracts the accessibility tree via the Chrome DevTools Protocol, filters out nodes with no accessible content, and serializes each remaining node in an indentation-based text format (`[id] role name value`). [6] We adopt this technique with three modifications.

First, each node is identified by a persistent identifier, the browser-assigned `backendDOMNodeId` or BrowserGym's `bid`, that remains stable across same-page DOM mutations. This enables stable cross-step node matching and serves as the basis for the observation transition history in Section 2.2. Second, BrowserGym unconditionally removes all property-less `generic` and `none` nodes, which causes wrapper elements that group related content in the DOM to collapse into flat sibling lists. We retain such a node when it has two or more child branches, each containing a visible descendant, preserving structural grouping that hierarchical decomposition relies on. Finally, for `image` and `link` nodes whose accessible name is empty, the node is enriched with the corresponding `src` or `href` attribute retrieved from the DOM.

E Training Dataset Construction

E.1 Source Page Collection

Domain categories. We select 10 domain categories from the 37 Tier 1 categories in the IAB Content Taxonomy 3.1 for their relevance to web agent tasks, covering Shopping, Travel, Technology & Computing, Business and Finance, Education, Food & Drink, Real Estate, Careers, Entertainment, and Sports.

Website selection. We use the Tranco top-1M ranking list snapshot from April 1, 2026 as the source. To assign each website to a domain category, we embed the 37 Tier 1 categories as reference embeddings and embed each website's concatenated title and description metadata using `sentence-transformers/paraphrase-MiniLM-L6-v2`. [7] Each website is assigned to the nearest category by cosine similarity. From the resulting clusters, we retain the 10 categories defined above and select the 500 highest-ranked websites by Tranco position across these categories.

Page URL sampling. For each website, page URLs are sampled from its `sitemap.xml` file. Each URL is scored by the sum of three signals from the sitemap metadata, namely priority (0.0–1.0, default 0.5), change frequency (0.15 for daily or hourly, 0.1 for weekly), and URL depth (0.03 per path segment, up to 5 levels). Up

[5] `https://github.com/imeanai/webcanvas?tab=readme-ov-file#download`

[6] `https://github.com/ServiceNow/BrowserGym/blob/main/browsergym/core/src/browsergym/core/observation.py`

[7] `https://huggingface.co/sentence-transformers/paraphrase-MiniLM-L6-v2`

to 100 URLs with the highest scores are retained per website. Websites without an accessible `sitemap.xml` or unreachable via Playwright headless Chromium are excluded, removing 247 of the original 500. The AXTree of each remaining page is extracted via Playwright, yielding 21,974 pages from 253 websites.

E.2 Data Annotation

Since the knowledge of how web pages are functionally organized is implicit, we construct training data for both decomposition and abstraction using `gpt-5-mini-2025-08-27` as the annotator. Because decomposition produces the region partition that abstraction then interprets, any partition error contaminates downstream abstraction labels. We therefore add a verification stage between the two, retaining only pages where every region passes validation. The annotation accordingly proceeds through three stages.

Decomposition annotation. The annotator receives each page's preprocessed AXTree together with the page URL and produces a list of region root node IDs. Since the annotator occasionally assigns an entire page to a single region, a fallback mechanism re-partitions any region whose node count exceeds 50% of the page total and is more than 10 times the median region size. Of the 21,974 pages, 7 fail due to context length limits and 2,690 (12.2%) trigger the fallback. The remaining 21,967 pages yield 547,075 regions, averaging 24.9 per page.

Partition verification. The annotator receives each page's region partition and identifies regions that were incorrectly decomposed. Since even a single invalid region is sufficient to corrupt the abstraction labels derived from it, we retain only pages that yield a valid region partition, one in which every region is correctly decomposed. This reduces 21,967 pages to 2,052 (9.3%) with 46,487 regions, averaging 22.7 per page.

Abstraction annotation. The annotator receives each verified region's AXTree subtree and produces a purpose and a state summary. Of the 46,487 regions, 1,340 consist solely of `none` or `generic` nodes with no visible content and are excluded, yielding 45,147 annotated regions from 2,052 pages. The annotations reduce the average region representation from 176.6 tokens to 56.2 tokens under the OpenAI `o200k_base` tokenizer, resulting in a 68.2% reduction.

Table 3 summarizes the dataset at each stage of the construction process. The annotation prompts used at each stage are provided in Figure 9, 10, and 11, respectively.

Table 3: Statistics at each stage of training dataset construction.

Domain category	# Sites	# Source pages	# Verified pages	# Edges	# Cut edges	# Regions
Technology & Computing	97	8,329	674 (32.85%)	234,708	15,366 (6.55%)	15,677
Entertainment	34	3,103	254 (12.38%)	141,090	9,363 (6.64%)	9,428
Sports	34	3,028	291 (14.18%)	91,487	6,676 (7.30%)	6,861
Business and Finance	25	2,293	274 (13.35%)	50,644	4,836 (9.55%)	4,968
Shopping	19	1,632	266 (12.96%)	17,471	1,823 (10.43%)	1,986
Education	19	1,523	102 (4.97%)	26,764	2,488 (9.30%)	2,535
Real Estate	10	807	63 (3.07%)	17,581	1,158 (6.59%)	1,209
Travel	7	509	29 (1.41%)	5,208	574 (11.02%)	565
Careers	6	600	44 (2.14%)	24,655	1,503 (6.10%)	1,535
Food & Drink	2	150	55 (2.68%)	7,346	329 (4.48%)	383
Total	253	21,974	**2,052** (100%)	**616,954**	**44,116** (7.15%)	**45,147**

F Region4Web Implementation Details

F.1 Hierarchical Decomposition

Node features. The feature vector $\mathbf{x}_v$ is 16-dimensional, concatenating a learned role embedding (11 dimensions) with five numeric features. The role vocabulary contains 204 entries, 203 from the Chromium accessibility role enumeration [8] and one for unknown roles. The five numeric features are the node's depth in the tree, subtree size, number of children, accessible name presence, and child role diversity, providing structural cues beyond the role embedding for boundary classification. Accessible name presence is set to 1 when the node has a non-empty accessible name and 0 otherwise. Child role diversity calculates the ratio of unique child roles to the number of children.

[8] Chromium 125.0.6422.26. `chromium/src/ui/accessibility/ax_enums.mojom`

Model architecture. REGIONENCODER and EDGECLASSIFIER are both three-layer MLPs with ReLU activations and a hidden dimension of 256. REGIONENCODER maps a 272-dimensional input ($\mathbf{x}_v$ and the merged children aggregation) to the 256-dimensional representation $\mathbf{r}_v$. EDGECLASSIFIER maps a 528-dimensional input ($\mathbf{x}_v$, $\mathbf{r}_{c_i}$, and $\bar{\mathbf{r}}$) to a scalar logit $\hat{y}_{v,c_i}$. The model totals approximately 536K parameters including the role embedding table. The full inference procedure is given in Algorithm 1.

Algorithm 1 Hierarchical Decomposition

Require: Page AXTree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$, threshold τ
Ensure: Region partition $\mathcal{R}$

$\mathcal{R} \leftarrow \emptyset$
for each node $v \in \mathcal{V}$ in bottom-up order **do**
  $S_v \leftarrow \{v\}$
  $\mathbf{x}_v \leftarrow \text{Concat}(\mathbf{E}_{\text{role}}(v),\ \mathbf{n}_v)$
  if v is a leaf **then**
    $\mathbf{r}_v \leftarrow \text{REGIONENCODER}(\mathbf{x}_v,\ \mathbf{0})$
  else
    Let $c_1, \ldots, c_k$ be the children of v
    $\bar{\mathbf{r}}_v \leftarrow \frac{1}{k} \sum_{j=1}^{k} \mathbf{r}_{c_j}$ ▷ Sibling mean
    $\mathcal{M}_v \leftarrow \emptyset$
    for $i = 1, \ldots, k$ **do**
      if $\text{EDGECLASSIFIER}(\mathbf{x}_v,\ \mathbf{r}_{c_i},\ \bar{\mathbf{r}}_v) \geq \tau$ **then**
        $\mathcal{R} \leftarrow \mathcal{R} \cup \{S_{c_i}\}$ ▷ Cut: c_i's subtree constitutes a region
      else
        $\mathcal{M}_v \leftarrow \mathcal{M}_v \cup \{c_i\}$
        $S_v \leftarrow S_v \cup S_{c_i}$ ▷ Merge: c_i's subtree merges into v's region
      end if
    end for
    if $\mathcal{M}_v \neq \emptyset$ **then**
      $\mathbf{r}_v \leftarrow \text{REGIONENCODER}\big(\mathbf{x}_v,\ \frac{1}{|\mathcal{M}_v|} \sum_{c_j \in \mathcal{M}_v} \mathbf{r}_{c_j}\big)$
    else
      $\mathbf{r}_v \leftarrow \text{REGIONENCODER}(\mathbf{x}_v,\ \mathbf{0})$
    end if
  end if
end for
$\mathcal{R} \leftarrow \mathcal{R} \cup \{S_{v_{\text{root}}}\}$ ▷ Root subtree constitutes the final region
return $\mathcal{R}$

Training configuration. The model is trained with teacher forcing, where ground-truth edge labels determine cut and merge decisions during the bottom-up traversal rather than the model's own predictions. Training runs for 140 epochs on a NVIDIA RTX A6000 GPU with Adam optimizer at a learning rate of 1×10^{-4} and gradient clipping at 1.0, and focal loss with $\alpha = 0.75$ and $\gamma = 2.0$ to address the class imbalance between merge and cut edges. The data is split into 90% training and 10% validation sets at the page level with seed 42.

Checkpoint selection and threshold tuning. The training epoch and the inference threshold τ are determined in two steps, each using the metric that matches its objective.

The training epoch is selected based on edge-level F1 on the validation set, as the model directly optimizes edge-level binary classification during training. Among the epochs with the highest validation F1, we choose the one with the smallest training-validation F1 gap to avoid overfitting, yielding epoch 125. Figure 7 shows the edge-level F1 curves over training.

The inference threshold τ converts edge-level logits into a region partition, whose quality is not captured by edge-level metrics. We therefore tune τ at the region level. Each ground-truth region is matched to the predicted region with the highest Intersection-over-Union (IoU), and counted as matched if this IoU meets or exceeds 0.5. Region-level precision, recall, and F1 are then computed over the matched counts relative to the total predicted and ground-truth regions. This yields $\tau = 0.55$. Table 4 reports the region-level metrics across threshold values.

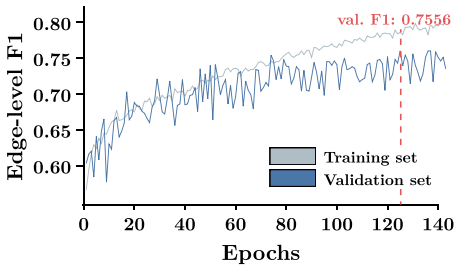


Figure 7: Edge-level F1 on training and validation sets over 140 epochs. Epoch 125 is selected for deployment.

Table 4: Region-level precision, recall, and F1 across inference thresholds at epoch 125. $\tau = 0.55$ achieves the highest F1.

τ	Precision	Recall	F1
0.35	0.5035	0.8440	0.6307
0.40	0.5704	0.8438	0.6806
0.45	0.6331	0.8315	0.7189
0.50	0.7185	0.8082	0.7607
0.55	**0.7755**	**0.7743**	**0.7749**
0.60	0.7964	0.6852	0.7366
0.65	0.8341	0.6139	0.7073
0.70	0.8589	0.5396	0.6628

F.2 Semantic Abstraction

Training configuration. `Qwen3-0.6B` is fine-tuned with full supervised fine-tuning in `bfloat16` precision with gradient checkpointing. Each training example pairs a region's preprocessed AXTree subtree as input with a JSON object containing the corresponding purpose p_i and state summary s_i as output, using the annotation prompt as the instruction prefix, which is shown in Figure 11. The loss is computed only on the output tokens, with all input and padding tokens masked. Training runs for 90 epochs (76,200 steps) on 3 NVIDIA RTX A6000 GPUs using distributed data parallel (DDP) [19] with a per-device batch size of 1 and gradient accumulation of 16, yielding an effective batch size of 48. The optimizer is AdamW with a learning rate of 5×10^{-6}, 200 linear warmup steps, and cosine decay. The maximum sequence length is 8,192 tokens, and 37 samples (0.08%) exceeding this limit are skipped during training. The data is split into 90% training and 10% validation sets with the same seed 42 used throughout decomposition model training.

Checkpoint selection. The checkpoint at step 65,350 is selected by jointly considering validation loss and manual quality assessment of sampled outputs. Figure 8 shows the training and validation loss curves.

Inference. The fine-tuned model processes regions with greedy decoding. The annotation prompt as in Figure 11 is reused at inference to maintain distributional consistency between training and deployment.

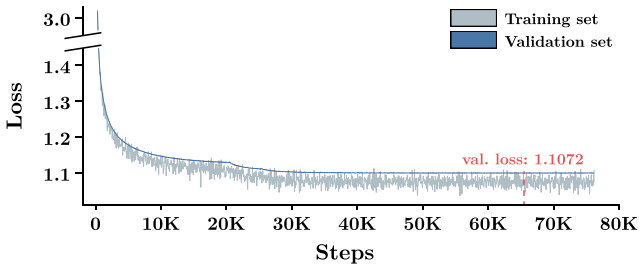


Figure 8: Training and validation loss over 90 epochs. Step 65,350 is selected for deployment.

G Case Studies

We provide qualitative case studies of Region4Web's decomposition and abstraction stage on representative pages from each WebArena domain in Tables 5 through 9.

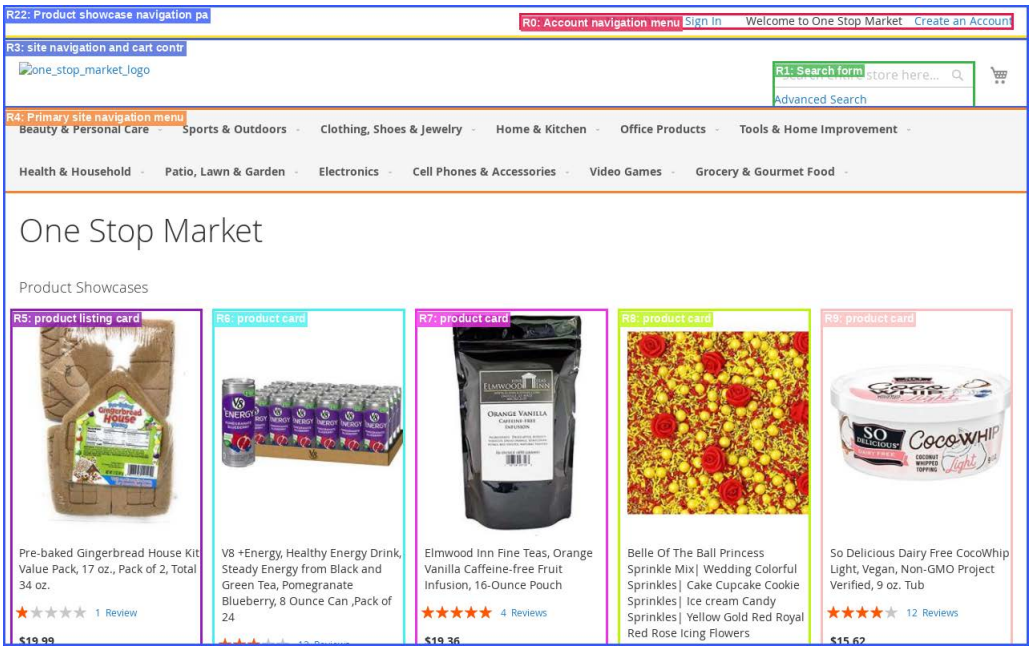
H Prompts

For reproducibility, we provide all prompts used in this work in Figures 9 through 14.

Table 5: Region4Web output on WebArena Shopping domain.

Page URL: http://localhost:7770 (343 nodes)

Hierarchical Decomposition: total 23 regions



Semantic Abstraction: R0, R1, R5, R7, R8

R0 (**purpose**: Account navigation menu)
state summary: Provides navigation links to account-related pages (My Account, My Wish List, Sign In, Create an Account) and a Welcome message. The Create an Account link is actionable to initiate a new account.

R1 (**purpose**: Search form)
state summary: Search is currently enabled with a combobox labeled "Search" and a "Advanced Search" link. The combobox is not expanded and the button is disabled.

R5 (**purpose**: product listing card)
state summary: Product: Pre-baked Gingerbread House Kit Value Pack, 17 oz., Pack of 2, Total 34 oz. with a 20% rating and $19.99. Available actions: Add to Cart, Add to Wish List, and Add to Compare.

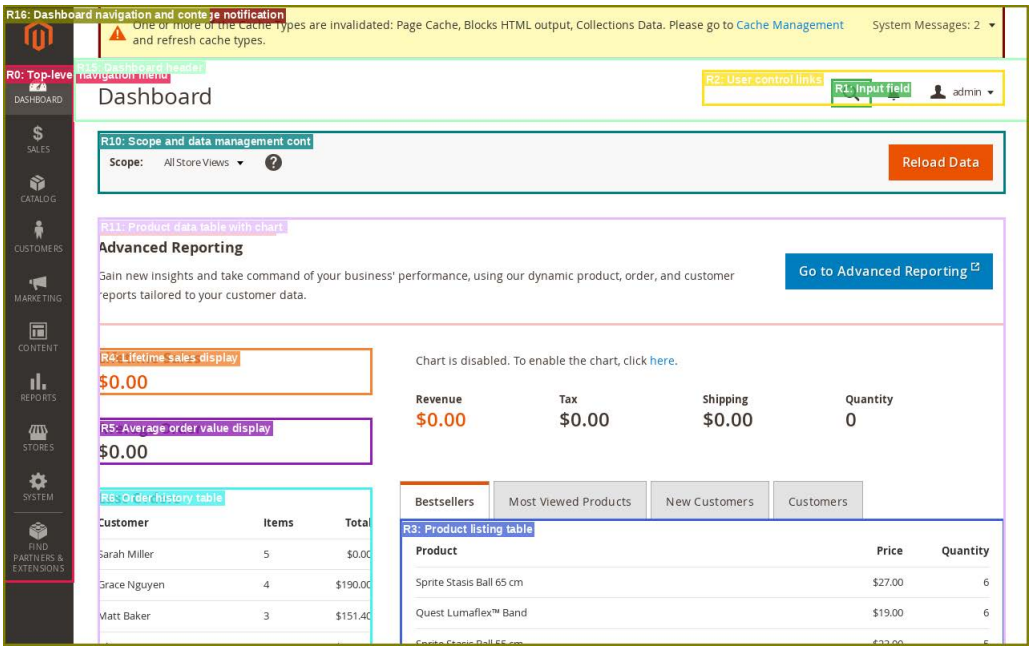
R6 (**purpose**: product card)
state summary: Healthy energy drink with a 57% rating and $14.47 price. Available actions: Add to Cart, Add to Wish List, and Add to Compare.

R7 (**purpose**: product card)
state summary: Product: Elmwood Inn Fine Teas, Orange Vanilla Caffeine-free Fruit Infusion, 16-Ounce Pouch (95% rating) priced at $19.36. Available actions: Add to Cart, Add to Wish List, and Add to Compare.

Table 6: Region4Web output on WebArena CMS (shopping admin) domain.

Page URL: http://localhost:7780/admin/admin/dashboard (217 nodes)

Hierarchical Decomposition: total 17 regions



Semantic Abstraction: R2, R5, R6, R10

R2 (**purpose**: User control links)
state summary: Contains two clickable links labeled "admin"s. Clicking either link navigates to the corresponding admin page.

R5 (**purpose**: Average order value display)
state summary: Shows an average order value of $0.00. No interactive controls are present in this region.

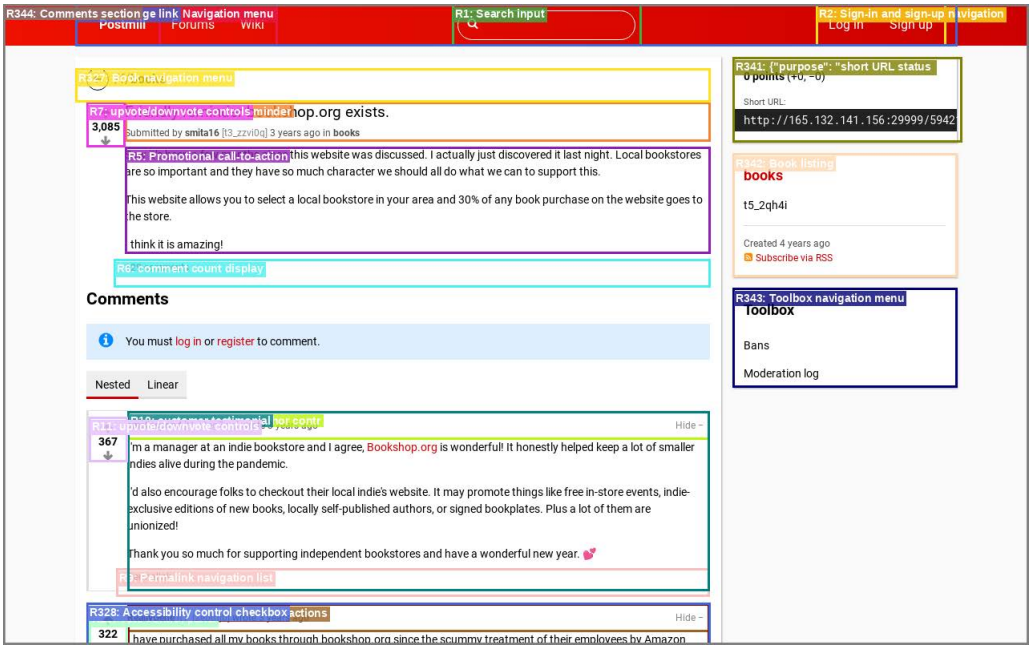
R6 (**purpose**: Order history table)
state summary: Shows order details for five orders (ID 299, 65, 125, 136, 230) with each row showing customer name, item count, and total. Each order link is actionable (clickable URL) to view the order.

R10 (**purpose**: Scope and data management controls)
state summary: Shows a 'Scope:' heading and provides a 'All Store Views' button with a menu popup and a 'Reload Data' button. The 'What is this?' link is actionable for clarification.

Table 7: Region4Web output on WebArena Reddit domain.

Page URL: http://localhost:9999/friedly-reminder-bookshop-org-exists (4,151 nodes)

Hierarchical Decomposition: total 345 regions



Semantic Abstraction: R5, R6, R11, R342

R5 (**purpose**: Promotional call-to-action)
state summary: Promotes a local bookstore program that 30% of book purchases go to the store and encourages supporting local bookstores. The region contains static text and a closing statement that appears to be a call to action.

R6 (**purpose**: comment count display)
state summary: Shows a count of 129 comments. The item is a link that can be activated to open the comment list or view more details.

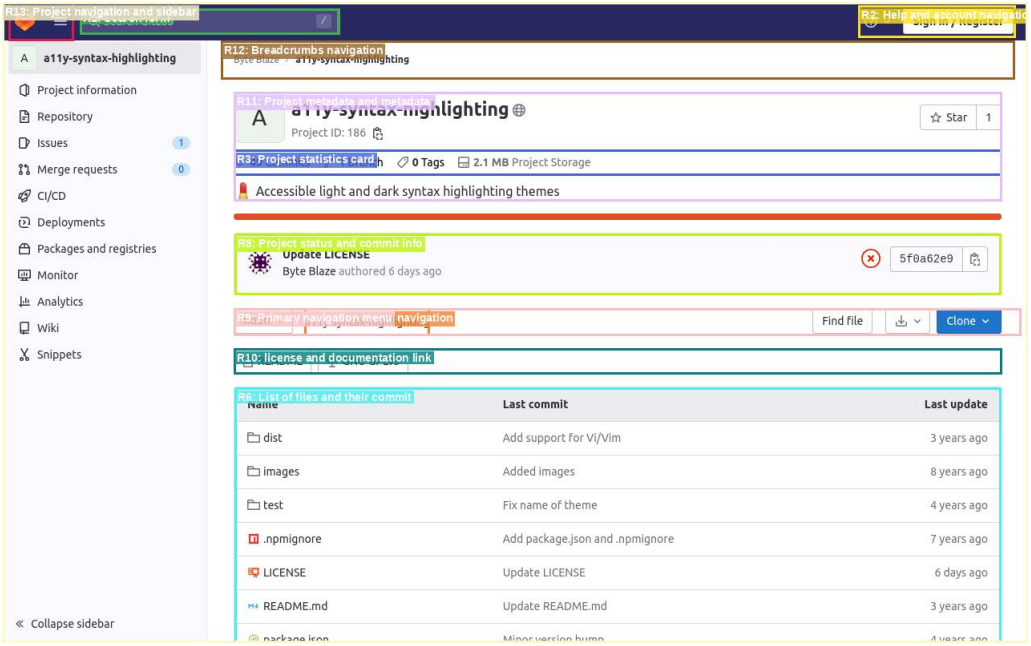
R11 (**purpose**: upvote/downvote controls)
state summary: Contains two buttons labeled "Upvote" and "Downvote" with a numeric value of 367 displayed. Clicking the buttons will toggle the up/down vote state and the 367 number is static text showing the current count.

R342 (**purpose**: Book listing)
state summary: Contains a single book entry with a 'books' link and a 'Subscribe via RSS' image. The book's timestamp shows it was created 4 years ago.

Table 8: Region4Web output on WebArena Gitlab domain.

Page URL: http://localhost:8023/byteblaze/a11y-syntax-highlighting (546 nodes)

Hierarchical Decomposition: total 14 regions



Semantic Abstraction: R2, R3, R6, R10

R2 (**purpose**: Help and account navigation links)
state summary: Contains two links: a 'Help' link with an image and a 'Sign in / Register' link. Clicking either will navigate to the help documentation or account sign-in/register page.

R3 (**purpose**: Project statistics card)
state summary: Shows project statistics: 49 commits, 1 branch, 0 tags, and 2.1 MB project storage. All items are clickable links that navigate to the corresponding metrics.

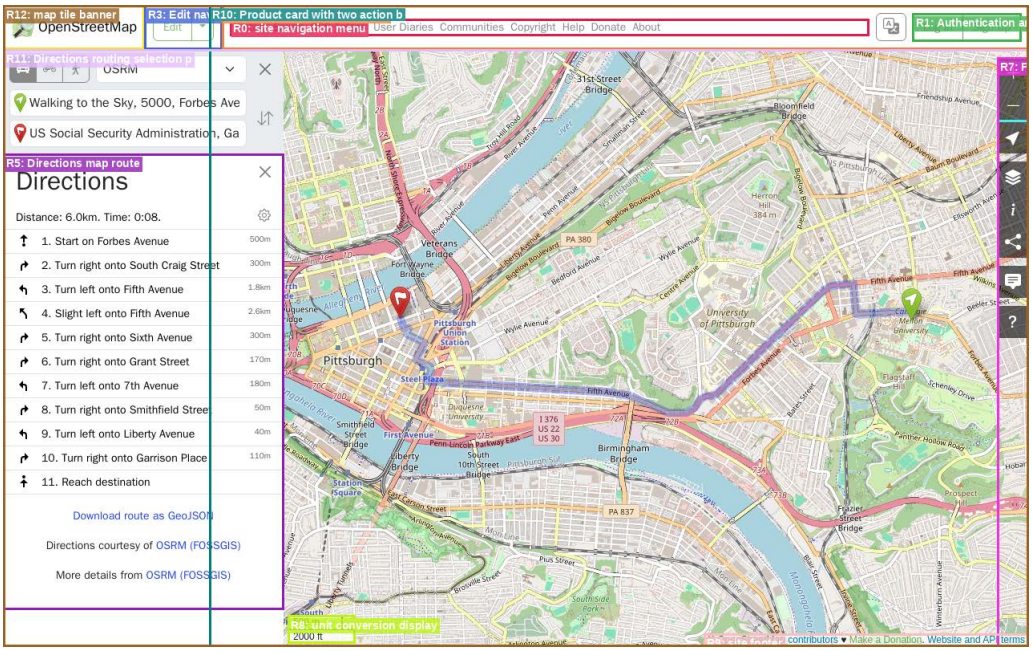
R6 (**purpose**: List of files and their commit/updates)
state summary: Contains a list of files (dist, images, test, LICENSE, README.md, package.json) with their last commit and update dates. Each file is a link that opens the file's page or shows the file's name and time.

R10 (**purpose**: license and documentation links)
state summary: Contains two clickable links: a README image and a GNU GPLv3 license link. Click either link to open the corresponding documentation or license page.

Table 9: Region4Web output on WebArena Map domain.

Page URL: https://www.openstreetmap.org/directions?engine=fossgis_osrm_car... (176 nodes)

Hierarchical Decomposition: total 13 regions



Semantic Abstraction: R0, R1, R5, R7, R11

R0 (**purpose**: site navigation menu)
state summary: Provides navigational links to site sections: History, Export, GPS Traces, User Diaries, Communities, Copyright, Help, Donate, and About. Each item is a clickable link to navigate to the corresponding page.

R1 (**purpose**: Authentication and sign-up navigation)
state summary: Provides two navigation links: 'Log In' to initiate account access and 'Sign Up' to create a new account. Both links are actionable and can be activated to proceed with the respective authentication or sign-up process.

R5 (**purpose**: Directions map route)
state summary: Provides a route map with 11 steps (1–11) and a destination. Includes a downloadable GeoJSON file and a link to the OSRM (FOSSGIS) source. The heading 'Directions' is a heading and the table shows distance and time for each step.

R7 (**purpose**: Page header controls)
state summary: Provides navigation links to Layers, Legend, Share, Add a note to the map, and Query features. A 'Show My Location' button is available to open the location view.

R11 (**purpose**: Directions routing selection panel)
state summary: Selects directions services (OSRM) and provides a 'Reverse Directions' button to reverse the route. The 'From' and 'To' fields are populated with the specified addresses and the 'Close' button cancels the panel.

Decomposition prompt

You are an observation space analyst for web agents. You partition a web page's accessibility tree (AXTree) into non-overlapping functional regions so that an autonomous agent can understand and interact with the page.

<definition>
A functional region is a subtree whose elements are collectively organized to serve a distinct purpose. Functional purpose is not a property of individual elements. It arises from how elements are collectively organized. A single link has an element-level action, but a region-level purpose emerges only when multiple elements are organized together to fulfill a coherent function. Every node belongs to exactly one region.
</definition>

<constraints>
1. Structural containers (the tree root, ARIA landmarks like banner/main/contentinfo) group content by page position, not by purpose. Always evaluate their children. A container becomes a region only for children that do not form their own regions.

2. A region must be meaningful to an agent, something it would need to independently recognize or interact with to carry out a task. Purely decorative elements and isolated utility shortcuts belong to their parent's region.
</constraints>

<algorithm>
To evaluate a node N, apply these steps in order.

Step 1. Container passthrough.
If N is the tree root or an ARIA landmark, evaluate each direct child of N by applying this algorithm recursively. N itself becomes a region only for its remaining children, those that did not form their own regions. If all children form regions, N is not recorded.

Step 2. Evaluate N.
For each candidate N that is not a container:

(a) If N's children are structurally repetitive (a collection of entries), determine whether each entry's purpose arises from its own internal organization: its own elements collectively organized into different roles (information, metadata, and actions) around a single entity, interpretable without shared context from siblings or parent structure. If yes, each entry is its own region. Apply this algorithm recursively to each. If no, N is one region. Record N.

(b) If N has multiple children that serve distinct purposes and each child's purpose arises from its own internal organization, each such child is its own region. Apply this algorithm recursively to each. Children whose purpose does not arise from their own internal organization remain in N's region.

(c) If neither (a) nor (b) applies, N serves one coherent purpose. Record N.

Step 3. Meaningfulness check.
Before recording N, verify it is meaningful to an agent (see constraint 2). If not, N belongs to its parent's region.
</algorithm>

<output_format>
Output the recorded region root node IDs as a comma-separated list.
After the list, output nothing further.
</output_format>

URL: {url}

AXTree:
{axtree}

Figure 9: Prompt for decomposition annotation stage.

Verification prompt

You are an observation space analyst for web agents. You verify whether a page's accessibility tree (AXTree) has been correctly partitioned into functional regions.

<definition>
A functional region is a subtree whose elements are collectively organized to serve a distinct purpose. Functional purpose is not a property of individual elements. It arises from how elements are collectively organized. A single link has an element-level action, but a region-level purpose emerges only when multiple elements are organized together to fulfill a coherent function.
</definition>

<criteria>
For each region, verify that it is correctly formed by checking:

1. The region corresponds to one recognizable functional unit on the page, an area that an agent would identify as serving a single role.

2. If you can identify multiple sub-components within the region that are each independently recognizable as their own functional area on the page, the region is incorrectly formed. Those areas should be separate regions.
</criteria>

<output_format>
Output the IDs of incorrectly formed regions as a comma-separated list (e.g., R3, R7).
If no region is incorrectly formed, output: none
After the output, output nothing further.
</output_format>

URL: {url}

{region_partition}

Figure 10: Prompt for partition verification stage.

Abstraction prompt

You are an observation space analyst for web agents. You produce semantic descriptions of functional regions extracted from web page accessibility trees (AXTree).

<task>
Given a region's AXTree subtree, produce two descriptions:

1. purpose: Identify the collective function that the region's elements are organized to serve. This should name the type of region, not describe its current contents or enumerate its features. Write a short noun phrase.

2. state_summary: Interpret the region's current content and available actions to inform task-based decision making. Lead with the key information an agent would match against a task, not with descriptions of what the region shows. Write one to two concise sentences.
</task>

<guidelines>
- Derive both fields solely from the elements present in the subtree.
- For purpose, identify what the elements collectively accomplish, not what individual elements are.
- For state_summary, interpret and select what matters for decision making, not exhaustively describe elements.
- Always output in English, translating non-English content as needed.
</guidelines>

{region_axtree}

Figure 11: Prompt for abstraction, used across annotation, training, and inference.

Selection prompt

You are a page region selector for a web agent. You receive the agent's task, the actions taken so far, and a list of functional regions on the current page, each described by its purpose and state summary. Select the regions the agent needs on this page to make progress on the task.

<principles>
1. First understand what the current page offers from the full set of region abstractions. Then, given the task and the action history, select every region whose content could be relevant to the task, whether the agent needs to interact with it or read information from it. Do not exclude regions based on an assumed course of action.

2. Exclude a region only when its purpose is clearly unrelated to the task. If relevance cannot be determined from the description, include the region.

3. When multiple regions share a similar purpose and their state summaries do not indicate which ones the task requires, include all of them.

4. A state summary that appears to match the task does not by itself justify excluding other potentially relevant regions. The rendered content may not satisfy the task's exact requirements.
</principles>

<output_format>
Output the selected region IDs as a comma-separated list (e.g., R3, R7).
After the list, output nothing further.
</output_format>

Task: {task_instruction}

Action history:
{action_history}

{region_abstractions}

Figure 12: Prompt for task-relevant region selection.

Action selection prompt

You are a web agent. You receive a task, the current page state, and your previous actions. You select the next action to make progress on the task.

<action_space>
{action_space}
<action_space>

Elements are identified by unique bid. Use bid to refer to elements in your actions. Interacting with comboboxes, dropdowns, and auto-complete fields may require different actions depending on the element. Try select_option first. If it does not work, try fill or click and observe the result. Your final message to the user must contain only the answer value in the format implied by the task, with no prefixes, explanations, or restatements.
</action_space>

<output_format>
Reason about the current state and decide the next action inside <think> tags.
Then output exactly one action inside <action> tags.
After the tags, output nothing further.

<think>
Your step-by-step reasoning.
</think>
<action>
click('a324')
</action>
</output_format>

Task: {task_instruction}

Action history:
{action_history}

{axtree}

Figure 13: Prompt template for action selection. {action_space} is replaced with the set of 15 available actions and their descriptions provided by BrowserGym. (e.g., `click`, `fill`)

Action selection prompt (w/ PageDigest)

# action space
view_all()
  Description: Reorganize the current page state into functional regions and reveal all of them in their full AXTree form. Use this when the currently exposed regions appear insufficient for the task, for example when the target element seems missing from the observation or when no available action makes progress. This action remains in effect until the agent navigates to a new page.
  Examples:
    view_all()

observation space
<observation_space>
The page is split into functional regions, each rendered as a <Rx purpose="..."> ... </Rx> block. Regions deemed task-relevant render their full AXTree subtree inside the block; others appear as the opening tag with no inner content. Newly appeared nodes since the page was first entered are listed at the end inside an <added_elements> block.
If the elements you need to interact with do not appear inside any rendered subtree, or no available action makes progress, call view_all() to re-expose every region's full subtree for the rest of this page.
</observation_space>

Figure 14: Prompt for action selection with PageDigest. Only the additions to Figure 13 are shown.